

```
In [9]: #Load and Explore the Dataset
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
df=pd.read_csv(r"C:/Users/Administrator/Desktop/UST/Heart Disease UCI.csv")
df
```

```
Out[9]:
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	tl
0	69	1	0	160	234	1	2	131	0	0.1	1	1	
1	69	0	0	140	239	0	0	151	0	1.8	0	2	
2	66	0	0	150	226	0	0	114	0	2.6	2	0	
3	65	1	0	138	282	1	2	174	0	1.4	1	1	
4	64	1	0	110	211	0	2	144	1	1.8	1	0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...
292	40	1	3	152	223	0	0	181	0	0.0	0	0	
293	39	1	3	118	219	0	0	140	0	1.2	1	0	
294	35	1	3	120	198	0	0	130	1	1.6	1	0	
295	35	0	3	138	183	0	0	182	0	1.4	0	0	
296	35	1	3	126	282	0	2	156	1	0.0	0	0	

297 rows × 14 columns



```
In [6]: #Display first 10 rows
df.head(10)
```

```
Out[6]:
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	69	1	0	160	234	1	2	131	0	0.1	1	1	C
1	69	0	0	140	239	0	0	151	0	1.8	0	2	C
2	66	0	0	150	226	0	0	114	0	2.6	2	0	C
3	65	1	0	138	282	1	2	174	0	1.4	1	1	C
4	64	1	0	110	211	0	2	144	1	1.8	1	0	C
5	64	1	0	170	227	0	2	155	0	0.6	1	0	2
6	63	1	0	145	233	1	2	150	0	2.3	2	0	1
7	61	1	0	134	234	0	0	145	0	2.6	1	2	C
8	60	0	0	150	240	0	0	171	0	0.9	0	0	C
9	59	1	0	178	270	0	2	145	0	4.2	2	0	2



```
In [2]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 297 entries, 0 to 296
Data columns (total 14 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   age         297 non-null   int64
 1   sex         297 non-null   int64
 2   cp          297 non-null   int64
 3   trestbps    297 non-null   int64
 4   chol        297 non-null   int64
 5   fbs         297 non-null   int64
 6   restecg     297 non-null   int64
 7   thalach     297 non-null   int64
 8   exang       297 non-null   int64
 9   oldpeak     297 non-null   float64
10  slope       297 non-null   int64
11  ca          297 non-null   int64
12  thal        297 non-null   int64
13  condition   297 non-null   int64
dtypes: float64(1), int64(13)
memory usage: 32.6 KB

```

```

In [11]: #Check data types
         print(df.dtypes)

```

```

age          int64
sex          int64
cp           int64
trestbps     int64
chol         int64
fbs          int64
restecg      int64
thalach      int64
exang        int64
oldpeak      float64
slope        int64
ca           int64
thal         int64
condition    int64
dtype: object

```

```

In [12]: #Find missing values
         df.isna()

```

Out[12]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope
0	False	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...
292	False	False	False	False	False	False	False	False	False	False	False
293	False	False	False	False	False	False	False	False	False	False	False
294	False	False	False	False	False	False	False	False	False	False	False
295	False	False	False	False	False	False	False	False	False	False	False
296	False	False	False	False	False	False	False	False	False	False	False

297 rows × 14 columns



In [12]:

```
#Display summary statistics
df.describe()
```

Out[12]:

	age	sex	cp	trestbps	chol	fbs	restecg
count	297.000000	297.000000	297.000000	297.000000	297.000000	297.000000	297.000000
mean	54.542088	0.676768	2.158249	131.693603	247.350168	0.144781	0.996000
std	9.049736	0.468500	0.964859	17.762806	51.997583	0.352474	0.994000
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000000
25%	48.000000	0.000000	2.000000	120.000000	211.000000	0.000000	0.000000
50%	56.000000	1.000000	2.000000	130.000000	243.000000	0.000000	1.000000
75%	61.000000	1.000000	3.000000	140.000000	276.000000	0.000000	2.000000
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.000000



In [16]:

```
#Gender Distribution Analysis
#Count number of males and females
a=df["sex"][df["sex"]==1]
a.count()
```

Out[16]: 201

In [17]:

```
b=df["sex"][df["sex"]==0]
b.count()
```

Out[17]: 96

```
In [27]: df.sex.sum()
```

```
Out[27]: 201
```

```
In [32]: c=len(df.sex)
c
```

```
Out[32]: 297
```

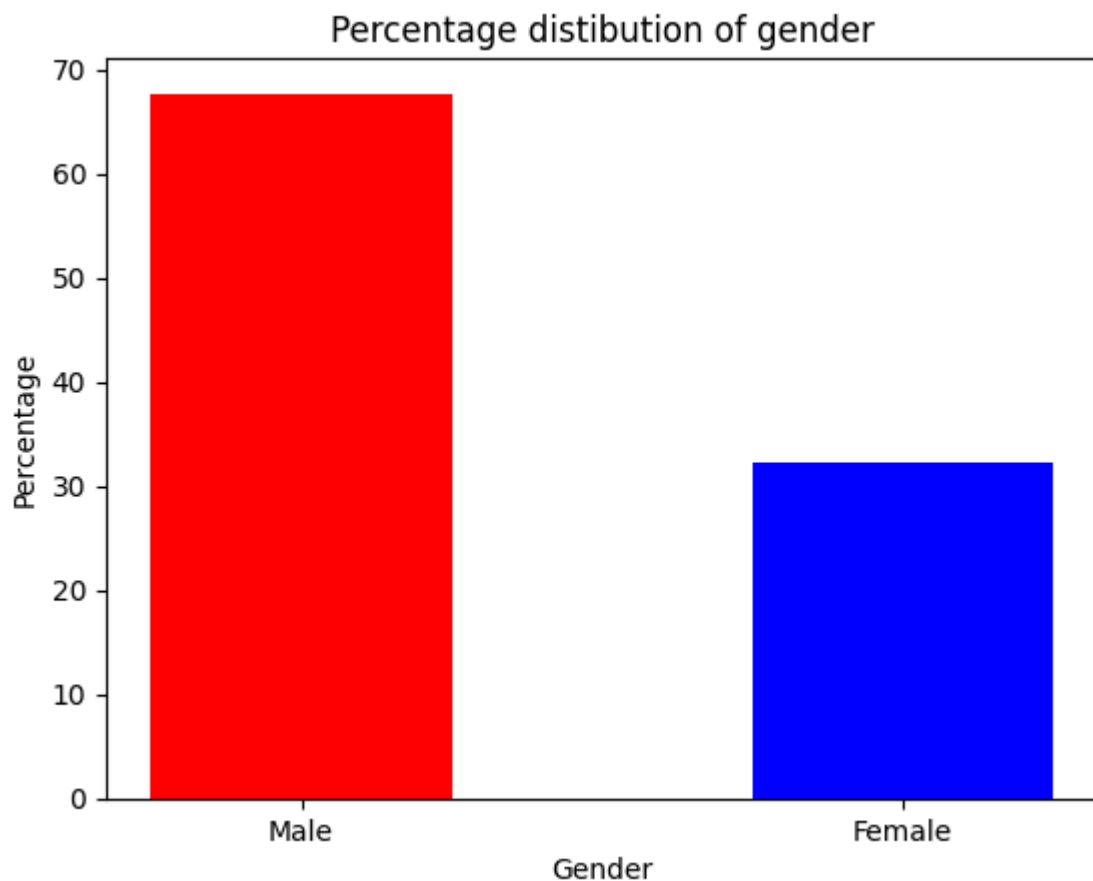
```
In [37]: #Calculate percentage distribution
mcount=(a.count()/c)*100
mcount
```

```
Out[37]: 67.67676767676768
```

```
In [38]: fcount=(b.count()/c)*100
fcount
```

```
Out[38]: 32.323232323232325
```

```
In [61]: #Plot a bar chart using Matplotlib
ca=["Male","Female"]
per=[mcount,fcount]
plt.bar(ca,per,color=["red","blue"],width=0.5)
plt.xlabel("Gender")
plt.ylabel("Percentage")
plt.title("Percentage distribution of gender")
plt.show()
```



```
In [46]: #Age Analysis
m1=df.age.min()
```

```
m1
```

```
Out[46]: 29
```

```
In [47]: m2=df.age.max()  
m2
```

```
Out[47]: 77
```

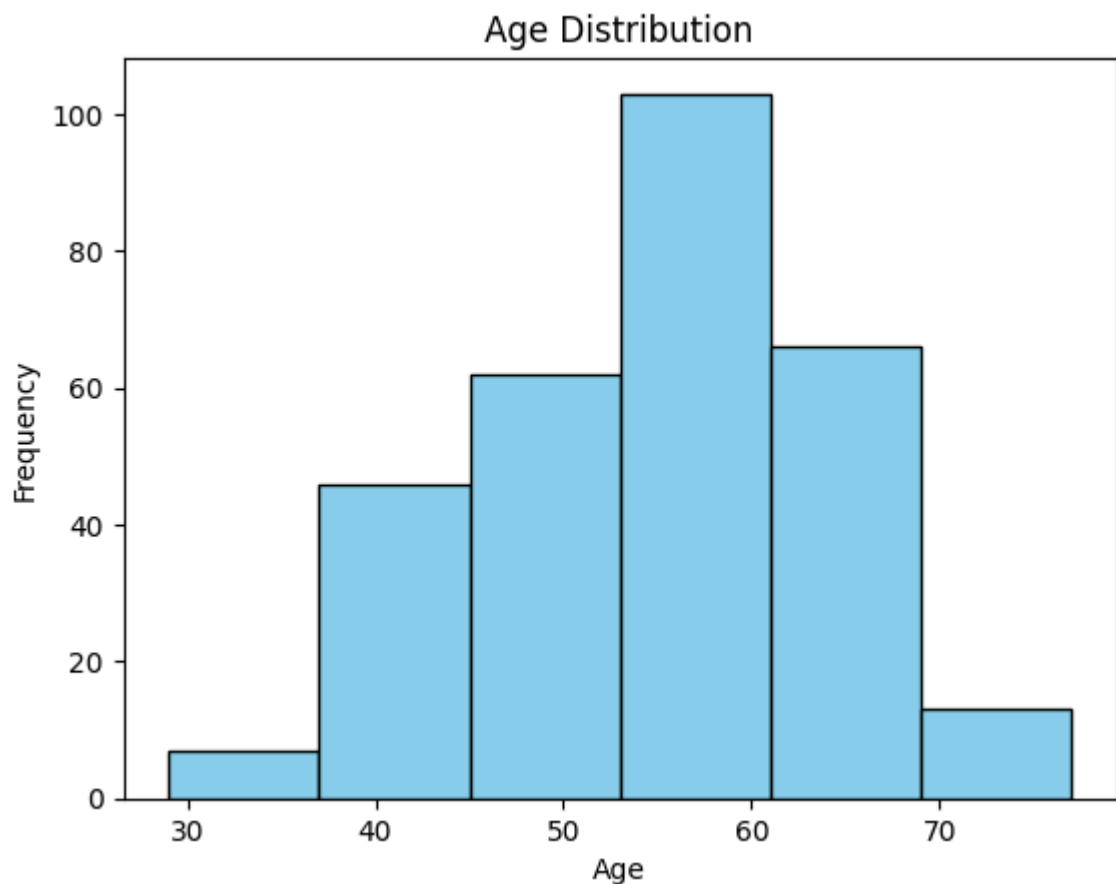
```
In [49]: m3=df.age.mean()  
m3
```

```
Out[49]: 54.54208754208754
```

```
In [50]: m4=df.age.median()  
m4
```

```
Out[50]: 56.0
```

```
In [60]: histo=[m1,m2,m3,m4]  
plt.hist(df.age, bins=6, color='skyblue', edgecolor='black')  
plt.xlabel("Age")  
plt.ylabel("Frequency")  
plt.title("Age Distribution")  
plt.show()
```



```
In [53]: #Target Variable Analysis  
#Count the n. patients with and without heart disease  
h1=df["condition"][df["condition"]==1]  
h1.count()
```

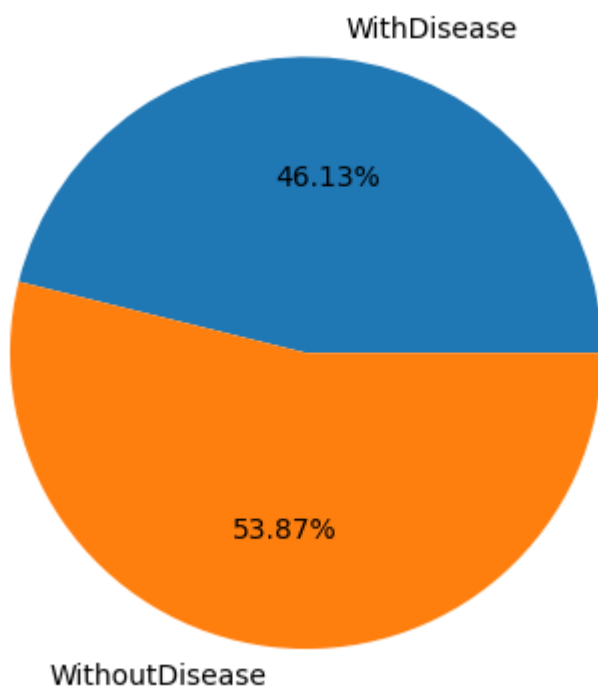
Out[53]: 137

```
In [55]: h2=df["condition"][df["condition"]!=0]
h2.count()
```

Out[55]: 160

```
In [57]: c1=["WithDisease","WithoutDisease"]
heart=[h1.count(),h2.count()]
plt.pie(heart,labels=c1,autopct='%2.2f%%')
```

```
Out[57]: ([<matplotlib.patches.Wedge at 0x1729db7ca10>,
<matplotlib.patches.Wedge at 0x1729db7c950>],
[Text(0.1334788514343031, 1.0918715108563732, 'WithDisease'),
Text(-0.13347885143430374, -1.0918715108563732, 'WithoutDisease')],
[Text(0.07280664623689259, 0.5955662786489309, '46.13%'),
Text(-0.07280664623689294, -0.5955662786489309, '53.87%')])
```



```
In [63]: d=len(df.condition)
d
```

Out[63]: 297

```
In [64]: d1=(h1.count()/d)*100
d1
```

Out[64]: 46.12794612794613

```
In [65]: d2=(h2.count()/d)*100
d2
```

Out[65]: 53.87205387205387

```
In [17]: # Correlation
corr_matrix = df.corr()
```

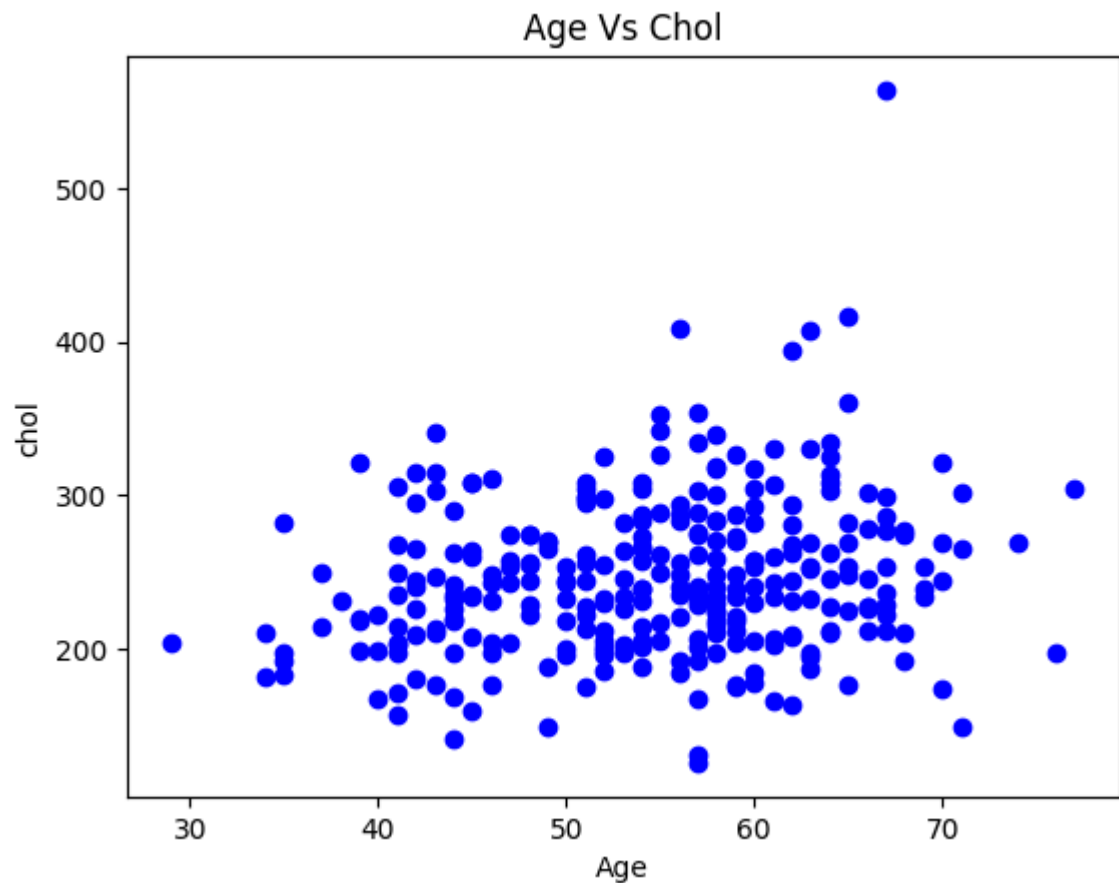
```
correlation = corr_matrix.loc["age", "chol"]
print(correlation)
```

0.2026435458466271

```
In [18]: if correlation > 0.5:
          print("There is a strong positive correlation: cholesterol tends to increase v
        elif correlation > 0.1:
          print("There is a weak positive correlation: cholesterol slightly increases wi
        elif correlation < -0.5:
          print("There is a strong negative correlation: cholesterol tends to decrease v
        elif correlation < -0.1:
          print("There is a weak negative correlation: cholesterol slightly decreases wi
        else: print("There is little to no correlation between age and cholesterol.")
```

There is a weak positive correlation: cholesterol slightly increases with age.

```
In [75]: plt.scatter(df["age"],df["chol"],color='blue')
          plt.xlabel("Age")
          plt.ylabel("chol")
          plt.title("Age Vs Chol")
          plt.show()
```



```
In [13]: #Chest Pain Type vs Disease
          #Group by cp and calculate disease rate
          group = df.groupby("cp")["condition"].mean()*100
          group
```

```
Out[13]: cp
0      30.434783
1      18.367347
2      21.686747
3      72.535211
Name: condition, dtype: float64
```

```
In [76]: d1
```

```
Out[76]: 46.12794612794613
```

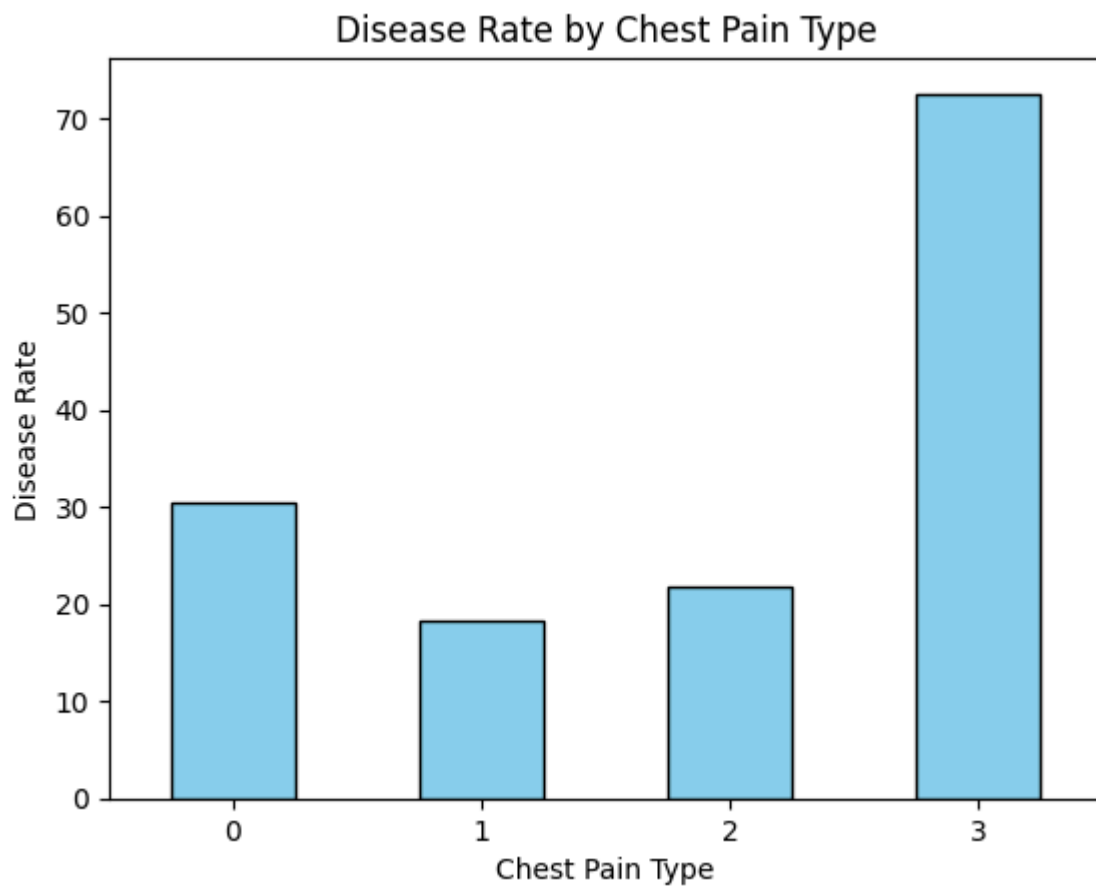
```
In [77]: d2
```

```
Out[77]: 53.87205387205387
```

```
In [16]: highest_value = group.max()
highest_value
```

```
Out[16]: 72.53521126760563
```

```
In [14]: group.plot(kind='bar', color='skyblue', edgecolor='black')
plt.title("Disease Rate by Chest Pain Type")
plt.xlabel("Chest Pain Type")
plt.ylabel("Disease Rate")
plt.xticks(rotation=0)
plt.show()
```



```
In [84]: #Average Cholesterol by Gender
s=df.groupby(df["sex"])
s
```

Out[84]: <pandas.core.groupby.generic.DataFrameGroupBy object at 0x000001729DD01DF0>

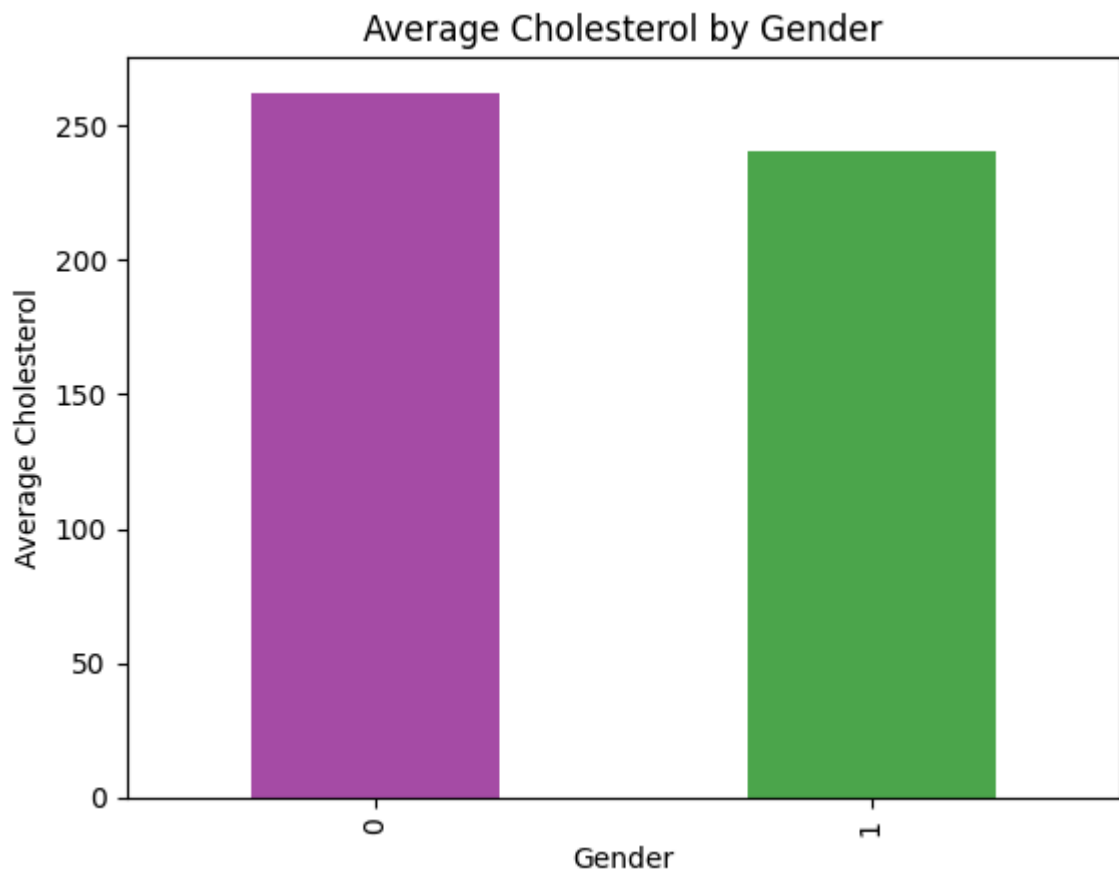
```
In [88]: c1=df.chol.mean()  
c1
```

Out[88]: 247.35016835016836

```
In [89]: avg_chol = df.groupby("sex")["chol"].mean()  
print(avg_chol)
```

```
sex  
0    262.229167  
1    240.243781  
Name: chol, dtype: float64
```

```
In [91]: import matplotlib.pyplot as plt  
  
avg_chol.plot(kind="bar", color=["purple", "green"], alpha=0.7)  
plt.xlabel("Gender")  
plt.ylabel("Average Cholesterol")  
plt.title("Average Cholesterol by Gender")  
plt.show()
```



```
In [97]: #Resting Blood Pressure Analysis  
s1=df.trestbps.sum()  
s1
```

Out[97]: 39113

```
In [99]: total=df.trestbps.count()  
total
```

Out[99]: 297

```
In [100]: avg=s1/total  
avg
```

Out[100]: 131.69360269360268

```
In [128]: highbp=df[df["trestbps"] > 140]  
bp  
highbp.count()
```

Out[128]: age 66
sex 66
cp 66
trestbps 66
chol 66
fbs 66
restecg 66
thalach 66
exang 66
oldpeak 66
slope 66
ca 66
thal 66
condition 66
dtype: int64

```
In [126]: Normalbp = df[df["trestbps"] <= 140]  
Normalbp.count()
```

Out[126]: age 231
sex 231
cp 231
trestbps 231
chol 231
fbs 231
restecg 231
thalach 231
exang 231
oldpeak 231
slope 231
ca 231
thal 231
condition 231
dtype: int64

```
In [129]: high_bp_rate = highbp["condition"].mean()  
normal_bp_rate = Normalbp["condition"].mean()
```

```
In [131]: #Maximum Heart Rate vs Disease  
#Compare average thalach for:  
#Disease patients  
avgthal=(df.thalach.sum())/(df.thalach.count())  
avgthal
```

Out[131]: 149.5993265993266

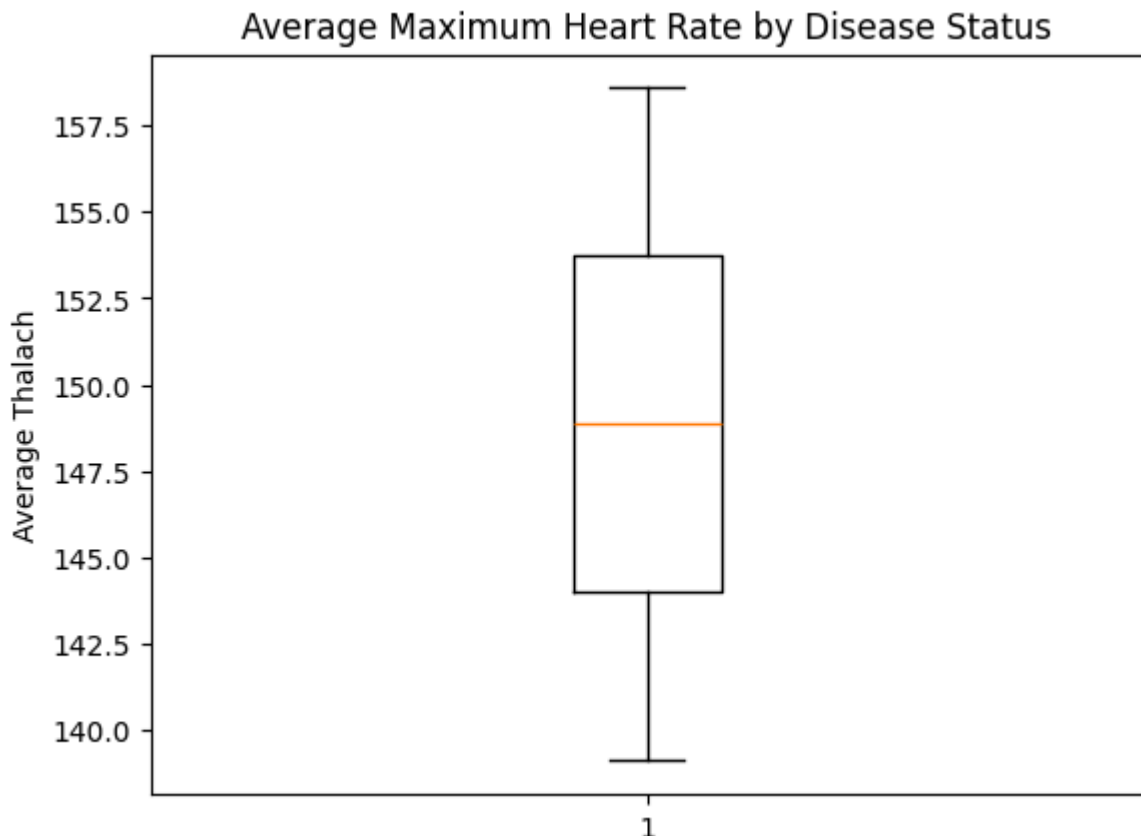
```
In [134]: diseasepatients = df[df["condition"] == 1]
```

```
no_diseasepatients = df[df["condition"] == 0]
```

```
In [135]: avgdisease = diseasepatients["thalach"].mean()
avg_no_disease = no_diseasepatients["thalach"].mean()
```

```
In [139]: values = [avgdisease, avg_no_disease]
labels = ["Disease", "No Disease"]
plt.boxplot(values)
plt.ylabel("Average Thalach")
plt.title("Average Maximum Heart Rate by Disease Status")
```

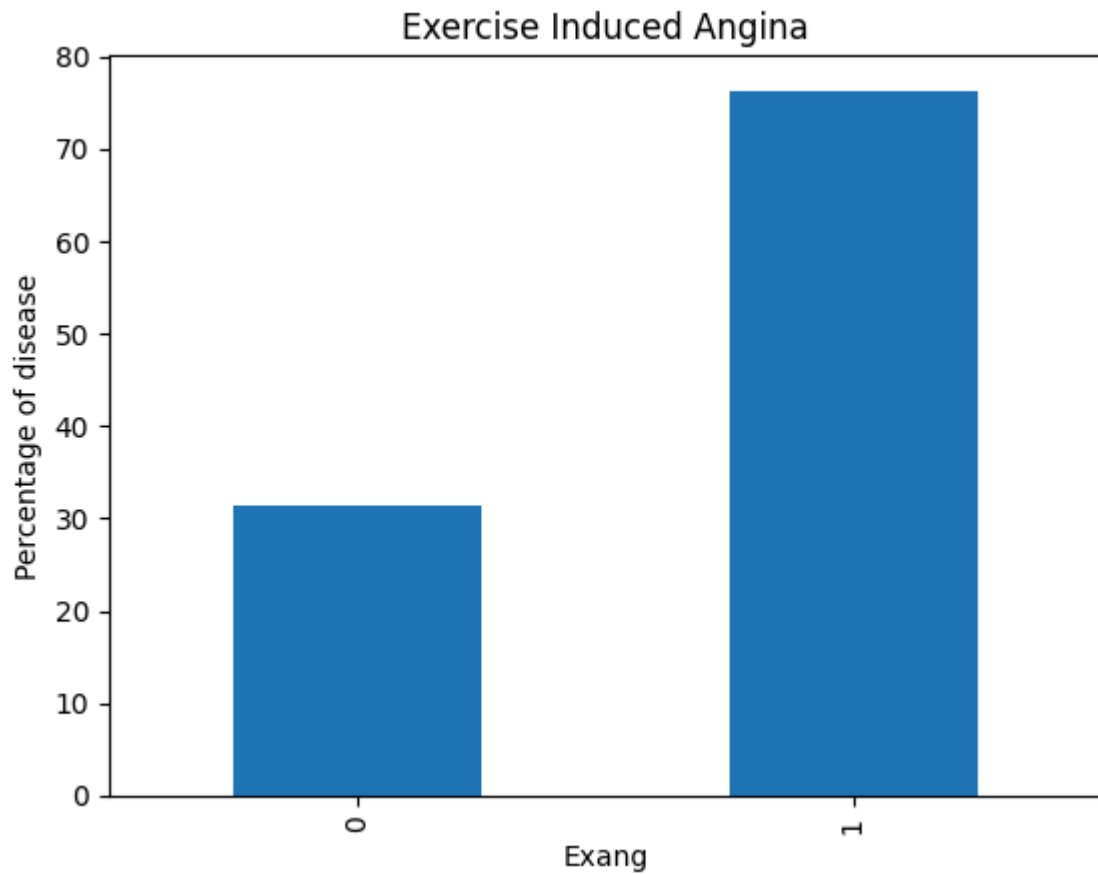
```
Out[139]: Text(0.5, 1.0, 'Average Maximum Heart Rate by Disease Status')
```



```
In [143]: #Exercise Induced Angina Impact
#Calculate disease percentage in:
exangper=df.groupby('exang')['condition'].mean()*100
exangper
```

```
Out[143]: exang
0      31.50000
1      76.28866
Name: condition, dtype: float64
```

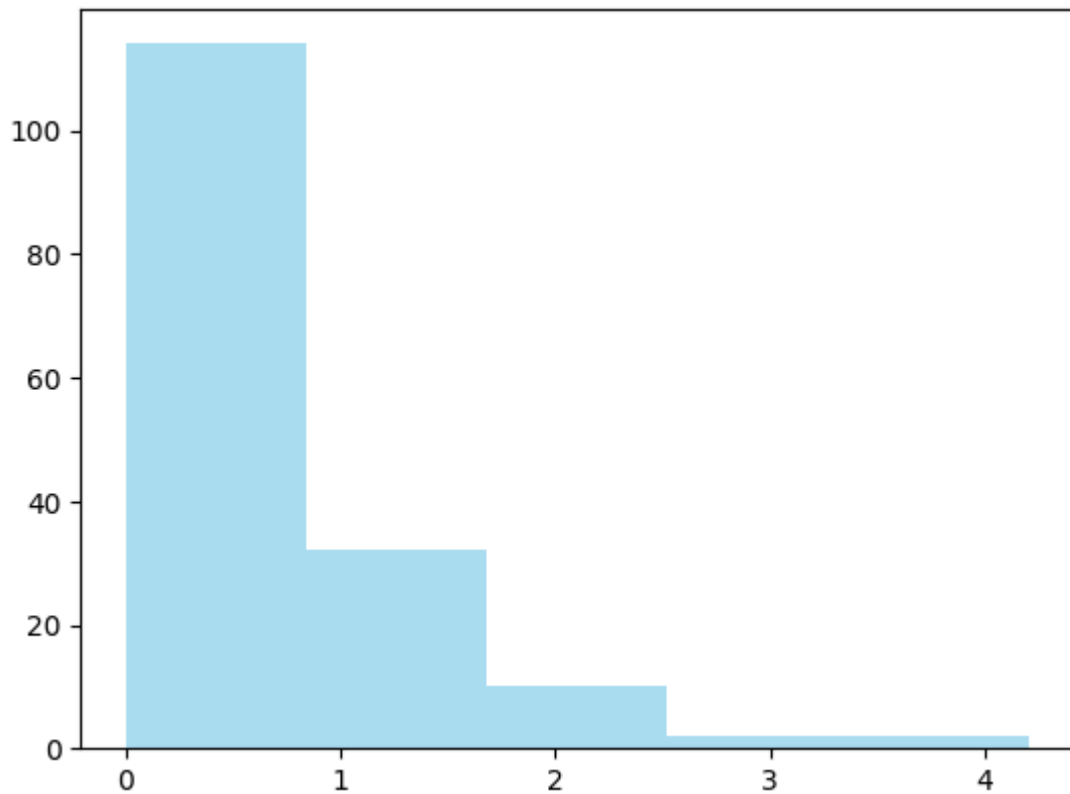
```
In [146]: exangper.plot(kind="bar")
plt.xlabel("Exang")
plt.ylabel("Percentage of disease")
plt.title("Exercise Induced Angina")
plt.show()
```



```
In [152]: #Calculate mean oldpeak by target  
#ST Depression (oldpeak) Analysis  
mean_oldpeak = df.groupby('condition')['oldpeak']
```

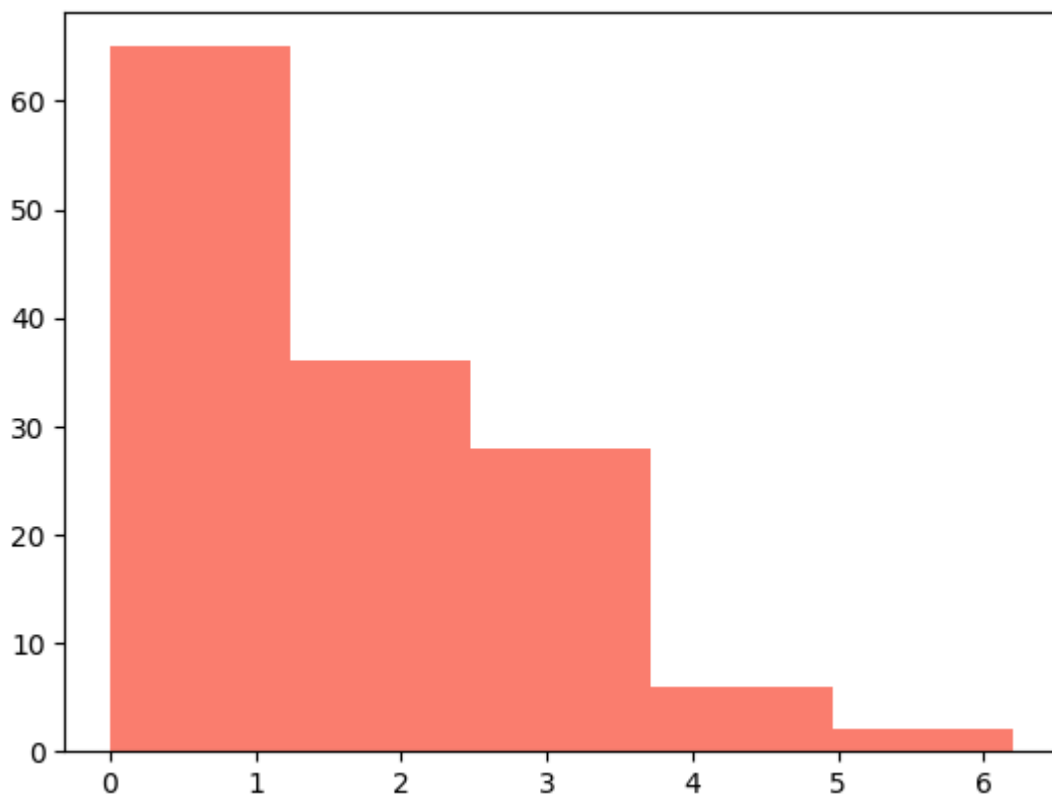
```
In [154]: plt.hist(df[df['condition']==0]['oldpeak'], bins=5, label='No Disease (condition=0)')
```

```
Out[154]: (array([114., 32., 10., 2., 2.]),  
          array([0. , 0.84, 1.68, 2.52, 3.36, 4.2 ]),  
          <BarContainer object of 5 artists>)
```



```
In [156]: plt.hist(df[df['condition']==1]['oldpeak'], bins=5, label='Disease (condition=1)',
```

```
Out[156]: (array([65., 36., 28., 6., 2.]),
          array([0. , 1.24, 2.48, 3.72, 4.96, 6.2 ]),
          <BarContainer object of 5 artists>)
```

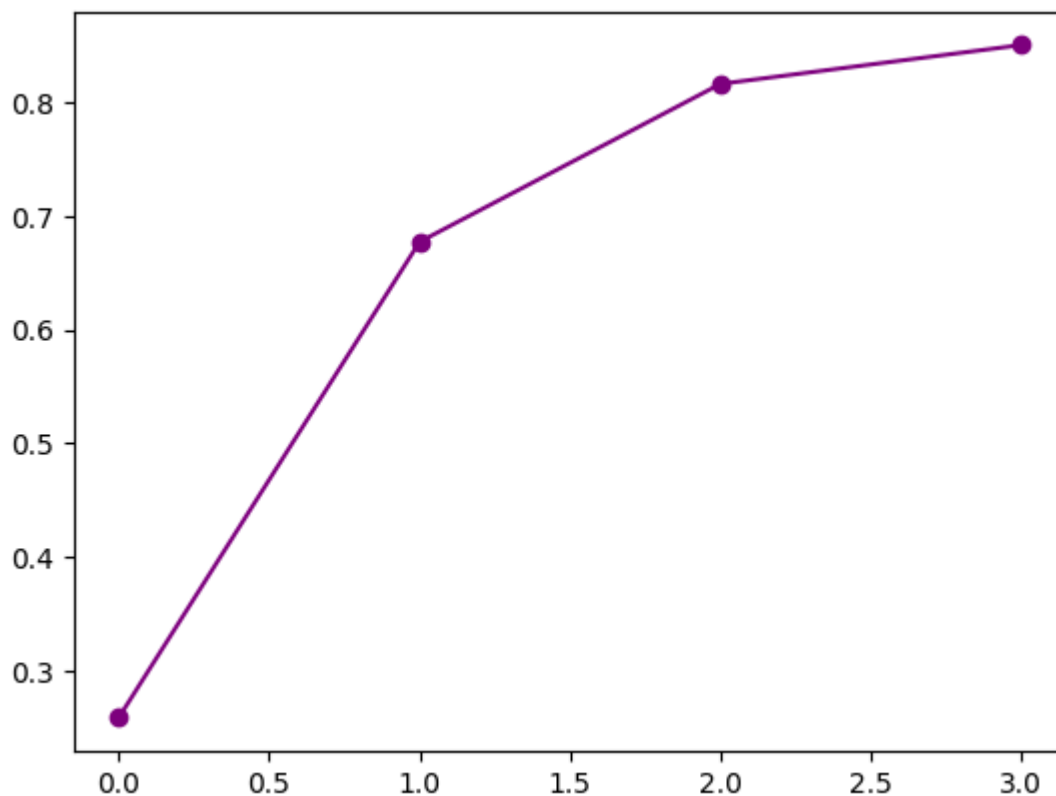


```
In [158]: #Number of Major Vessels (ca) Impact
disease_prob = df.groupby('ca')['condition'].mean()
disease_prob
```

```
Out[158]: ca
0      0.258621
1      0.676923
2      0.815789
3      0.850000
Name: condition, dtype: float64
```

```
In [159]: plt.plot(disease_prob.index, disease_prob.values, marker='o', linestyle='-', color
```

```
Out[159]: [<matplotlib.lines.Line2D at 0x172a021ed20>]
```



```
In [162]: #Thalassemia vs Disease
ct = pd.crosstab(df['thal'], df['condition'])
ct
```

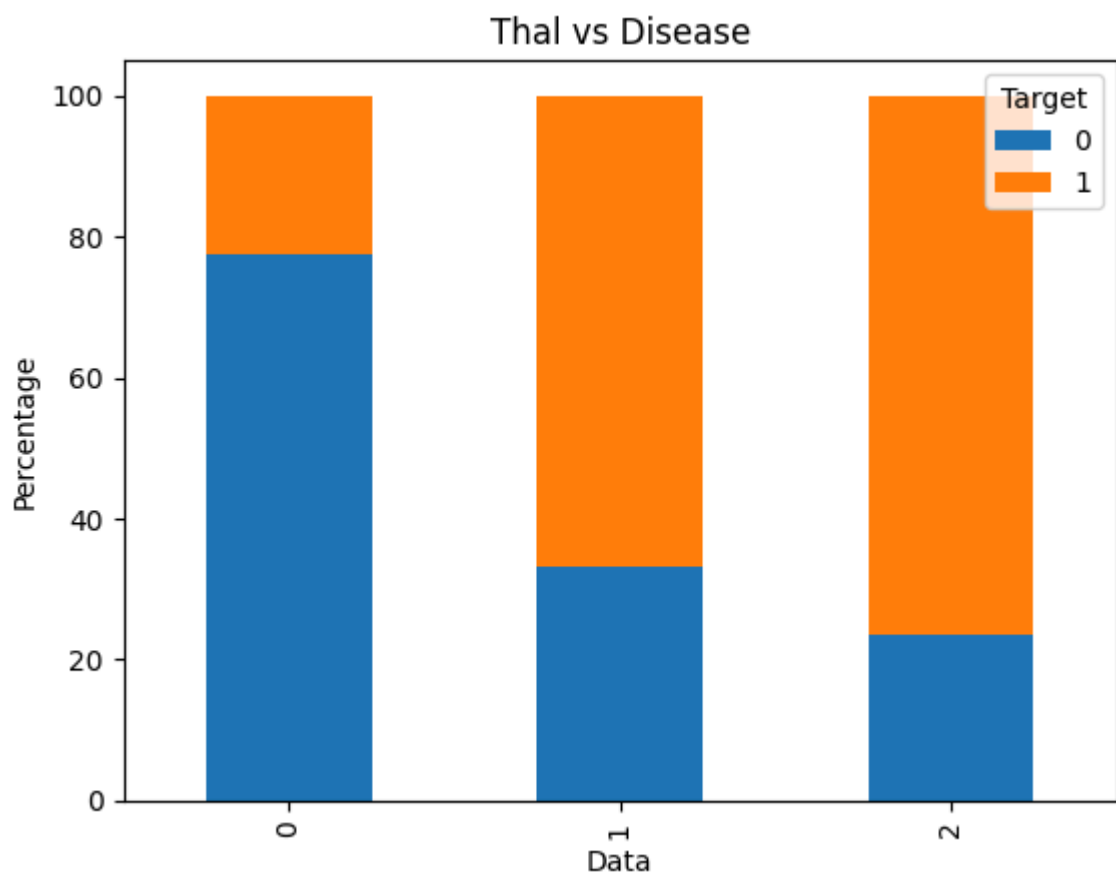
```
Out[162]: condition    0    1
          thal
          0  127  37
          1   6  12
          2  27  88
```

```
In [163]: ct_percent = ct.div(ct.sum(axis=1), axis=0) * 100
ct_percent
```

```
Out[163]: condition      0      1
```

	thal
0	77.439024 22.560976
1	33.333333 66.666667
2	23.478261 76.521739

```
In [167]: ct_percent.plot(kind='bar', stacked=True)
plt.xlabel("Data")
plt.ylabel("Percentage")
plt.title("Thal vs Disease")
plt.legend(title="Target")
plt.show()
```



```
In [171]: #Multi-Factor Risk Analysis
age = df['age'].values
chol = df['chol'].values
bp = df['trestbps'].values
target = df['condition'].values
```

```
In [180]: mas= (age > 50) & (chol > 240) & (bp > 140)
```

```
In [181]: selected = target[mas]
selected
```

```
Out[181]: array([0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1], dtype=int64)
```

```
In [182]: percentage_disease = (np.sum(selected == 1) / selected.size) * 100
percentage_disease
```

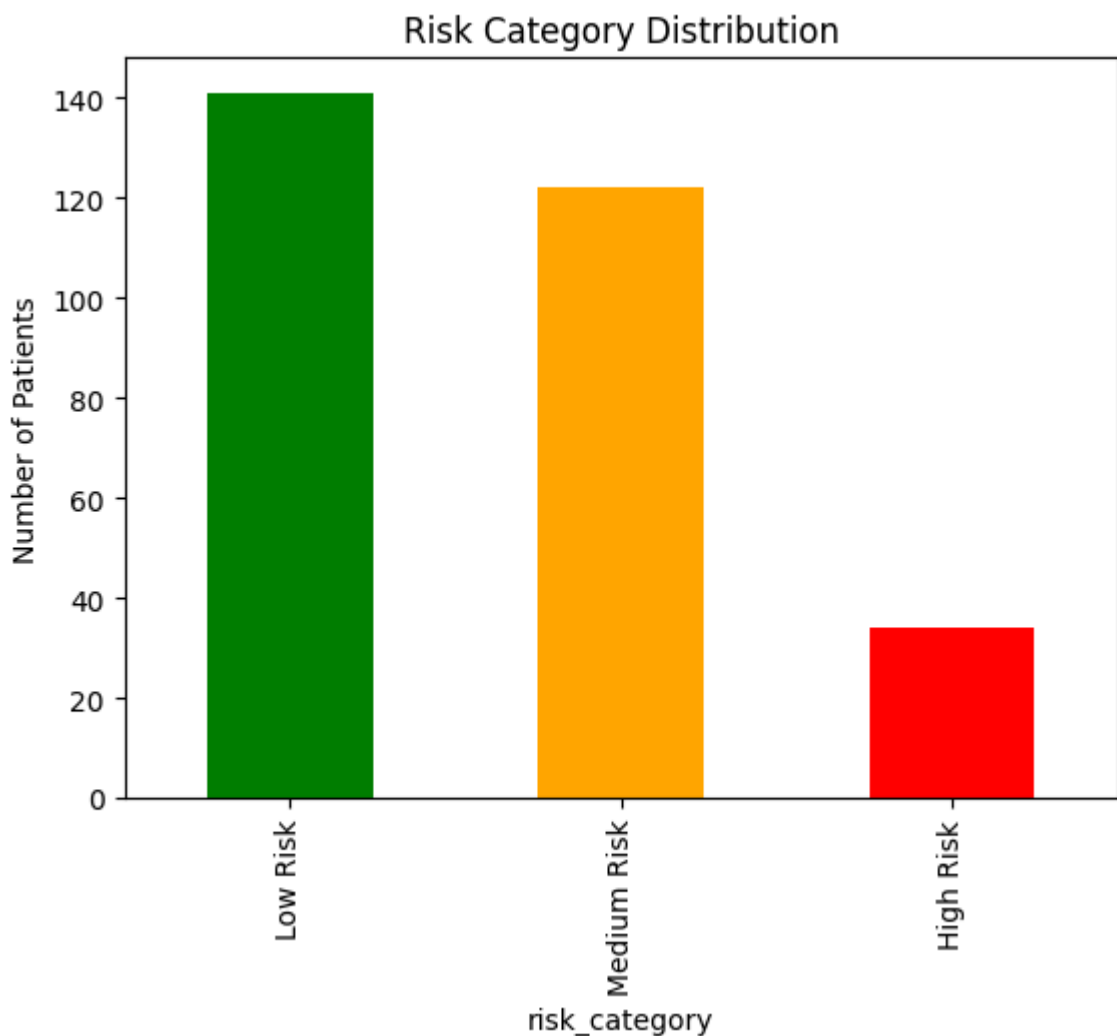
```
Out[182]: 66.66666666666666
```

```
In [183]: #Create Risk Score (Custom Analysis)
df['risk_score'] = (df['chol'] / 200) + (df['trestbps'] / 120) + df['oldpeak']
```

```
In [184]: def classify_risk(score):
    if score < 3:
        return "Low Risk"
    elif score < 5:
        return "Medium Risk"
    else: return "High Risk"

df['risk_category'] = df['risk_score'].apply(classify_risk)
```

```
In [185]: df['risk_category'].value_counts().plot(kind='bar', color=['green', 'orange', 'red'])
plt.ylabel("Number of Patients")
plt.title("Risk Category Distribution")
plt.show()
```



```
In [ ]:
```

```
In [ ]: #Does cholesterol strongly impact heart disease?
# Yes – high cholesterol is a major risk factor for heart disease.
```

- In []: *#Is male population more vulnerable?
#Yes – men are generally more vulnerable to heart disease than women,
#especially at younger ages.*
- In []: *#Does exercise-induced angina significantly increase risk?
#Yes – exercise-induced angina signals underlying
#coronary artery disease and significantly increases the risk of future heart prot*
- In []: *#Which feature has strongest correlation with disease?
#High blood pressure has the strongest correlation with heart disease,
#followed closely by smoking and high cholesterol*